

M10 Protected Review & Targeted Repair

OPEN ONLY AFTER a genuine attempt. Repair only the failed competency, then close this Review.

DE10-01 - Why containers exist

Expected reasoning: An image is packaged filesystem/config metadata; a container is a running instance that still uses the host kernel. Containers improve reproducibility, not magical security or persistence.

Target repair: If you call a container a VM, restate kernel sharing and lifecycle.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-02 - Images and containers

Expected reasoning: Build once, run many. Container changes disappear when the container is replaced unless persisted externally.

Target repair: If you rely on changes made interactively inside a container, rebuild the image instead.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-03 - Writing a Dockerfile

Expected reasoning: Cache-friendly build order copies dependency metadata before changing app code. Pin deliberately when reproducibility matters. Avoid secrets in build context/layers.

Target repair: If builds leak local files, inspect build context and .dockerignore.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-04 - Ports

Expected reasoning: EXPOSE documents intended container ports; it does not publish them. Host publication maps host:container and binding to 127.0.0.1 limits local exposure.

Target repair: If localhost cannot connect, separate app listen address, container port, published host port and firewall.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-05 - Volumes and persistence

Expected reasoning: Container writable layers are ephemeral. Volumes are engine-managed persistent stores; bind mounts expose host paths and fit different use cases.

Target repair: If data vanishes after recreation, prove where it was written.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-06 - Networks and service communication

Expected reasoning: Compose creates a project network by default; services on it resolve each other by service name. Do not hardcode changing container IPs.

Target repair: If service A cannot reach B, inspect network membership, service name and listening port before host port mappings.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-07 - Compose, configuration and secrets

Expected reasoning: Compose manages services, networks and volumes. Sensitive values should not be committed in source; use secrets or an appropriate secret backend.

Target repair: If a secret appears in Dockerfile, Git or logs, rotate it and redesign injection.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-08 - Why orchestration exists

Expected reasoning: Orchestration models dependencies, schedules, state, retries, observability and reruns. It coordinates work; it does not make unsafe tasks idempotent.

Target repair: If retrying the DAG duplicates data, repair task/data semantics before increasing retries.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-09 - Airflow architecture and UI

Expected reasoning: The scheduler decides runnable work; executors/workers run tasks; metadata tracks state; UI/API expose control/inspection. Airflow 3 uses a stable Task SDK/public interface.

Target repair: If UI says failed, read the specific task/run/log rather than restarting everything.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-10 - DAGs, tasks and dependencies

Expected reasoning: Tasks should have clear inputs/outputs, bounded side effects and explicit dependencies. Independent tasks can run concurrently when safe.

Target repair: If ordering is only hidden in code, make dependency structure explicit.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-11 - TaskFlow, operators and task communication

Expected reasoning: TaskFlow automatically wires returned values through Airflow task communication. XCom-style payloads are control/small metadata, not a replacement for data storage.

Target repair: If huge data is passed between tasks, persist it externally and pass a reference.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-12 - Scheduling and data intervals

Expected reasoning: A scheduled run represents a data interval, not simply the wall-clock instant the task starts. Use UTC-aware reasoning and explicit interval-based data queries.

Target repair: If a run reads now() blindly, backfills may process the wrong slice.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-13 - Retries, timeouts and failure behavior

Expected reasoning: Retry only plausibly transient work. A retry is safe only when side effects are idempotent/transactional. `execution_timeout` bounds a task attempt.

Target repair: If retries amplify damage, stop and repair idempotency or classify permanent errors.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-14 - Parameters and task communication

Expected reasoning: Params are configuration supplied/defaulted for a run and JSON-Schema validated. Task outputs carry produced metadata/results. Do not confuse them.

Target repair: If a parameter changes business scope, record it in run metadata.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-15 - Connections, variables and secrets

Expected reasoning: Connections identify external systems; secrets can come from environment/metastore/external backends. Masking helps but is not permission control.

Target repair: If plaintext credentials are committed, treat it as exposure and rotate.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-16 - Sensors and waiting

Expected reasoning: Sensors poll/wait. Reschedule or deferrable patterns can release worker resources; every wait needs a timeout/failure policy.

Target repair: If a sensor waits forever, add timeout and decide skip/fail/escalate behavior.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-17 - Logs and debugging

Expected reasoning: Start with failing task -> attempt -> exception -> input/config -> upstream state -> environment. Logs should include run/task identifiers but never secrets.

Target repair: If you rerun without identifying cause, you may hide an intermittent defect.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-18 - Catchup, backfills and reprocessing

Expected reasoning: Backfills recreate runs for past data intervals. Tasks must use interval inputs and idempotent writes. Catchup is a scheduling choice; backfill is an explicit operation.

Target repair: If historical rerun uses current date/config accidentally, stop and make interval/config explicit.

Fresh check: change one relevant input/configuration and re-prove the invariant.

DE10-19 - Integration and production-style orchestration

Expected reasoning: Production readiness combines reproducible runtime, clear dependencies, safe secrets, bounded retries/waits, idempotent data operations, reconciliation, logs and tested reprocessing.

Target repair: If a diagram cannot explain failure/recovery and evidence, it is not an operational design.

Fresh check: change one relevant input/configuration and re-prove the invariant.